

## Introduction

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleam animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine metu degendae praesidia firmissima. – Filium morte multavit. – Si sine causa, nollem me ab eo delectari, quod ista Platonis, Aristoteli, Theophrasti orationis ornamenta neglexerit. Nam illud quidem physici, credere aliquid esse minimum, quod profecto numquam putavisset, si a Polyaeno, familiari suo, geometrica discere maluisset quam illum etiam ipsum dedocere. Sol Democrito magnus videtur, quippe homini erudito in geometriaque perfecto, huic pedalis fortasse; tantum enim esse omnino in nostris poetis aut inertissimae segnitiae est aut fastidii delicatissimi. Mihi quidem videtur, inermis ac nudus est. Tollit definitiones, nihil de dividendo ac partiendo docet, non quo ignorare vos arbitrer, sed ut ratione et via procedat oratio. Querimus igitur, quid sit extremum et ultimum bonorum, quod omnium philosophorum sententia tale debet esse, ut eius magnitudinem celeritas, diuturnitatem allevatio consoletur. Ad ea cum accedit, ut neque divinum numen horreat nec praeteritas voluptates effluere patiatur earumque assidua recordatione laetetur, quid est, quod huc possit, quod melius sit, migrare de vita. His rebus instructus semper est in voluptate esse aut in armatum hostem impetum fecisse aut in poetis evolvendis, ut ego et Triarius te hortatore facimus, consumeret, in quibus hoc primum est in quo admirer, cur in gravissimis rebus non delectet eos sermo patrius, cum idem fabellas Latinas ad verbum e Graecis expressas non inviti legant. Quis enim tam inimicus paene nomini Romano est, qui.

# Physics

The branch of physics that we care about for this project are the Isaac Newton's classical mechanics and its laws of motion, I believe.

When I had to learn this at school, Wikipedia [1] and CrashCourse's YouTube's playlist on physics [2] made me understand mechanics.

Let's try to summarize the basics.

## Kinematic Equations

Every object has its *position*, like it's coordinates, represented by  $r$ .

When that object is in motion, there is a *velocity*,  $v$ , that is, the *speed* of the object in a certain *direction*. When there's no motion,  $v = 0$ . This speed and velocity are normally represented as the magnitude and direction — in Portuguese *sentido*, since *direção* alone doesn't let us represent the inverse of a vector — of a vector respectively.

Given a constant velocity (no acceleration)  $v$ , and  $\Delta t$  being the time passed, starting at  $t_0$  and ending at  $t_f$ ,  $r(\Delta t) = r(t_0) + vt_f$ <sup>1</sup>, so  $r$  is the position when the object moves with that velocity  $v$  for  $t$  units of time starting at position  $r(t_0)$ .

The average velocity is  $\bar{v} = \frac{\Delta r}{\Delta t}$ .<sup>2</sup> When we close that time window to a certain point of time, we tend to get the velocity at that point of time.<sup>3</sup> The velocity at given time  $t$  (instantaneous velocity) is represented as:

$$\begin{aligned} v(t) &= \lim_{\Delta t \rightarrow 0} \frac{\Delta r}{\Delta t} = \\ &= \lim_{\Delta t \rightarrow 0} \frac{r(t + \Delta t) - r(t)}{\Delta t} = \\ &= \frac{dr}{dt} = r'(t) \end{aligned}$$

Velocity changes position over time, and acceleration changes velocity over time.

So something similar happens to calculate the acceleration  $a$ :

$$\begin{aligned} a(t) &= \lim_{\Delta t \rightarrow 0} \frac{\Delta v}{\Delta t} = \\ &= \frac{dv}{dt} = v'(t) = \\ &= \frac{d^2 r}{dt^2} = r''(t) \end{aligned}$$

Because we don't like in a unidimensional world,  $v$  and  $a$  are represented in vectors:

$$\begin{aligned} \vec{a} &= \frac{\Delta \vec{v}}{t} \Leftrightarrow \vec{a}t = \Delta \vec{v} \Leftrightarrow \\ &\Leftrightarrow \vec{v} = \vec{v}_0 + \vec{a}t \end{aligned}$$

We can also use integrals to arrive to some common equations:<sup>4</sup>

---

<sup>1</sup>[3]

<sup>2</sup>[4]

<sup>3</sup>[5]

<sup>4</sup>[6]

$$\begin{aligned}
\overrightarrow{\Delta r} &= \int_{t_0}^t \vec{v} dt = \\
&= \vec{v}_0 t + \frac{\Delta \vec{v} t}{2} = \\
&= \vec{v}_0 t + \frac{\vec{a} t^2}{2}
\end{aligned}$$

That was assuming the acceleration is constant. From that, we arrive to the kinematic equation<sup>5</sup>

$$r = r_0 + \vec{v}_0 t + \frac{1}{2} \vec{a} t^2$$

### For Later

There are some equations that appear on the project statement that we can talk about right now.

We can calculate the new position of an object if we have a record of the of it's last its position, velocity and acceleration at a certain point of time. We then know how much time has passed, so from the last equation we get:

$$r(t + \Delta t) = r(t) + v(t)\Delta t + \frac{1}{2}a(t)\Delta t^2$$

We can do the same for the velocity:

$$v(t + \Delta t) = v(t) + a(t)\Delta t$$

On the project statement, the teacher suggests a *half-step* and finally a correction. I still have to figure out why exactly, but the logic is the same.

### Newton's Laws of Motion

1. For the velocity of an object to change, there must be a force acting upon it;
2. The linear momentum  $\vec{p} = m\vec{v}$  of the object will change depending on that force applied. All the forces acting upon the object at the moment, together, equal how much the linear momentum is changing at that time.

That is, assuming mass doesn't change over time:

$$\vec{F} = mv'(t) = m\vec{a}$$

3. If you hit a wall, the wall hits you back with equal force (and that's why it hurts).

So for any force, there's an equal one but with inverse *sentido*.

### Newton's Law of Universal Gravitation

This part about the history of the apple falling on Newton's head, we are learning together! Or at least I'll relearn something that didn't get stuck in my head this last year's I had physics in school.

Newton's law of universal gravitation describes gravity as a force by stating that every particle attracts every other particle in the universe with a force that is proportional to the product of their masses and inversely proportional to the square of the distance between their centers of mass.

---

<sup>5</sup>[7]

That gives the following:

$$F_{\text{gravity}} \propto \frac{m_a m_b}{\|\vec{r}_a - \vec{r}_b\|^2}$$

With empiricism, Newton figured out that the bigger the masses of the objects, the stronger the force; and the further apart the objects are from each other, the weaker the force.

Well, one thing is for sure: I am not some sort of magnet that when I'm walking around things just come towards me, and I only walk into a streetlight by the force of distraction, not by some sort of gravitational force. At the same time, when I fall, I find the ground and not the sky. Right. Newton came to the conclusion that the force between me and the Earth's surface (between me and the Earth really) is much, much greater than the force between me and... Imagine something really, really, big, and probably heavy, like a whale, or a huge building, the biggest in the world. That mass is nothing compared to the Earth's mass. But certainly there are heavier things than the Earth, right? Probably the Sun, for example. Yeah, probably, but don't forget that we are touching the Earth, and **the Sun is probably an astronomical unit away**, and that the distance between the objects also count.

Because of all the above, Newton added to the equation a constant  $G$  which represents just a very little and tiny number that would make those force calculation results seem more realistic.  $G$  initially didn't have a specific value attributed to it by Newton. Later scientists figured out that a good number for it is  $G = 6.6743015 \times 10^{-11} \text{m}^3 \text{kg}^{-1} \text{s}^{-2}$ .<sup>67</sup> As Newton figured out, pretty tiny. If we say  $\vec{\Delta r}_{ab}$  is the distance from object  $a$  to object  $b$ , the formula becomes:

$$\vec{F}_{\text{gravity}} = G \frac{m_a m_b}{\|\vec{\Delta r}\|^2} \frac{\vec{\Delta r}}{\|\vec{\Delta r}\|}$$

We get a force vector. We can also notice that the gravitational force from  $b$  to  $a$  is equal to that from  $a$  to  $b$ , but just the inverse vector (the third law of motion). Look, how cool is this! If you are free-falling, the force of gravity acting on you, based on the second law of motion, is:

$$\vec{F}_{\text{gravity}} = m\vec{g}$$

We are taught that the acceleration of Earth's gravity is close to  $\|\vec{g}\| = 9.8 \text{m/s}^2$ . Now look:<sup>8</sup>

$$\begin{aligned} \frac{F_{\text{gravity}}}{m_{\text{you}}} &= G \frac{m_{\text{Earth}}}{\|\vec{a} - \vec{b}\|^2} = \\ &= g = \\ &= 9.8 \end{aligned}$$

$\|\vec{a} - \vec{b}\|$  is the radius of the Earth<sup>9</sup> (I guess the Earth isn't a sphere, but you get the point), so:

$$\begin{aligned} G \frac{m_{\text{Earth}}}{6371008.7714^2} &= 9.8 \Leftrightarrow m_{\text{Earth}} = 9.8 \frac{6371008.7714^2}{G} \Leftrightarrow \\ &\Leftrightarrow m_{\text{Earth}} = 5.9598683 \times 10^{24} \text{ kg} \end{aligned}$$

I believe I did the maths right, and as you can see, the Earth is... heavy.

---

<sup>6</sup>[9]

<sup>7</sup>[10]

<sup>8</sup>[9]

<sup>9</sup>[11]

## $n$ -Body Problem

The problem of predicting the motion of  $n$  objects subject to gravity is known as the  $n$ -body problem.

— [8]

The problem itself isn't hard to understand, nor to implement. You obviously just calculate all of the accelerations to then apply to each object given  $\Delta t$ .

The problem then comes when the number of objects grow, because a calculation for the acceleration of an object depends on all other objects. Time complexity  $n(n-1) = O(n^2)$ , which doesn't look that crazy.

Literally, like it's shown in the project statement:

$$\begin{aligned}\vec{a}_i &= \frac{\vec{F}_i}{m_i} = \\ &= \frac{1}{m_i} \sum_{j=0, j \neq i}^n \vec{F}_{ij} = \\ &= \frac{Gm_i}{m_i} \sum_{j=0, j \neq i}^n \frac{m_j \vec{\Delta r}}{\|\vec{\Delta r}\|^3} = \\ &= G \sum_{j=0, j \neq i}^n \frac{m_j \vec{\Delta r}}{\|\vec{\Delta r}\|^3}\end{aligned}$$

The project statement actually introduces another variable,  $\varepsilon$ , which prevents from dividing by zero in case the two objects are in the exact same position. I guess that's not even possible in real life since there's flesh around my centre of gravity that prevents us from being in the same position, I guess that's unless something is part of me. To mimic real life, Each of the objects for the simulation will get a “radius” greater than zero, representing the “flesh”, the collision box. This will serve exactly like the *softening parameter*  $\varepsilon$ :

$$\begin{aligned}\because \text{radius}_a > 0 \wedge \text{radius}_b > 0 \\ \varepsilon &:= \text{radius}_a + \text{radius}_b \\ \therefore \varepsilon &> 0 \\ \therefore \forall x \in \mathbb{R}, \|\vec{\Delta r}\|^3 + \varepsilon &> 0 \\ \therefore \forall x \in \mathbb{R}, \|\vec{\Delta r}\|^3 + \varepsilon &\neq 0\end{aligned}$$

Division by zero won't happen this way:  $\varepsilon > 0 \Rightarrow \forall x \in \mathbb{R}, \|\vec{\Delta r}\|^3 + \varepsilon \neq 0$ .

Later, this idea of collisions can later serve the purpose to actually physically simulate them so that it becomes impossible for two or more objects to be “on top” of each other.

But now, where in the formula of acceleration would I put this? I think I just don't put it, and collisions should be taken care of programmatically and not with maths.

I might have just not understood fully how  $\varepsilon$  integrates on the formula, because, why are we doing the Euclidean distance between  $\|\Delta r\|$  and  $\varepsilon$ , to the power of 3, to substitute  $\|\Delta r\|$  alone, and what's

the logic behind it? Are there other solutions for the obvious problem? And also, is there an optimal value for  $\varepsilon$  then? I really don't get it.

I got it that we need it to “remove the singularity at small distances”<sup>10</sup>, but that's it. And by talking about optimal values for  $\varepsilon$ , I haven't even thought about the values for the other parameters. Maybe I'll just consider  $G = 1$  to simplify things, as I don't think I'll be simulating the cosmos.

What a *dilemma*!

## The singularity dilemma

$$\vec{a}_i = G \sum_{j=0, j \neq i}^n \frac{m_j \vec{\Delta r}}{\|\vec{\Delta r}\|^3}$$

When  $\vec{\Delta r} \rightarrow \vec{0}$ :

$$\begin{aligned} \lim_{\vec{\Delta r} \rightarrow \vec{0}} \vec{a}_i &= G \sum_{j=0, j \neq i}^n \left( \lim_{\vec{\Delta r} \rightarrow \vec{0}^+} \frac{m_j}{\|\vec{\Delta r}\|^2} \frac{\vec{\Delta r}}{\|\vec{\Delta r}\|} \right) \vee G \sum_{j=0, j \neq i}^n \left( \lim_{\vec{\Delta r} \rightarrow \vec{0}^-} \frac{m_j}{\|\vec{\Delta r}\|^2} \frac{\vec{\Delta r}}{\|\vec{\Delta r}\|} \right) = \\ &= G \sum_{j=0, j \neq i}^n \frac{\pm \infty \vec{\Delta r}}{\|\vec{\Delta r}\|} \\ &= \pm \infty \end{aligned}$$

I mean, you can imagine that because the closer two objects are to each other, the stronger the gravitational force is, there becomes a point that they are so close together that that force is infinitely big. I just can't fathom what happens to the direction of that vector in this case.

So that's another problem I did not think of earlier. We should “cap” huge speeds, but do we just reuse the last trajectory?

The project statement replaces  $\|\vec{\Delta r}\|$  with  $\sqrt{\|\vec{\Delta r}\|^2 + \varepsilon^2}$ . The logic is that there is always a *distance* that  $\varepsilon$  can “simulate” to be lesser than — but not 0, as that would destroy the purpose of it — that makes this substitution always greater than 0 and still be able to make  $\|\vec{\Delta r}\| \approx \sqrt{\|\vec{\Delta r}\|^2 + \varepsilon^2}$  so that the acceleration calculated is close to reality, which only happens when  $\varepsilon$  is way, way smaller than  $\|\vec{\Delta r}\|$ :

$$\begin{aligned} \therefore r &:= \|\vec{\Delta r}\|, r \geq 0, \forall \varepsilon \in \mathbb{R}, \sqrt{r^2 + \varepsilon^2} = 0 \Leftrightarrow r = \varepsilon = 0 \\ \therefore \forall \zeta > 0, \exists \delta = \delta(\zeta, r), \delta > 0, \forall \varepsilon \in \mathbb{R}, 0 < |\varepsilon| < \delta &\rightarrow \left| \sqrt{r^2 + \varepsilon^2} - r \right| < \zeta \end{aligned}$$

Here,  $\delta$  represents the distance that  $\varepsilon$  can't be greater than, and  $\delta$  depends on all the real distances between all objects  $\|\vec{\Delta r}_i\|$ <sup>11</sup>, but also on how much we want the gravitational force calculation to be close to reality, represented by  $\zeta$ . So if we want  $\zeta$  accuracy/precision/tolerance in our physics,  $\therefore |\varepsilon| < \delta(\zeta, \{x : \|\vec{\Delta r}_x\|\})$ . The gravitational force stops being accurate to what would happen in reality once  $\|\vec{\Delta r}\| < |\varepsilon|$ , which can only be an accurate value, again, only if  $|\varepsilon|$  is way, way smaller than  $\|\vec{\Delta r}\|$ .

Now we do not have the problem of dividing by 0 any more, but we have an idea of what the value of  $\varepsilon$  is based on (we think), but still don't know how to exactly calculate it. It's about when we stop

<sup>10</sup>[12]

<sup>11</sup>The thing is that, distances between objects change, but I don't know if  $\varepsilon$  is supposed to change or just to be a constant from the start to the end of the simulation.

caring about the real physics that are being applied, but we don't know what number we would like it to be for each possible universe of objects and their states. We might just start by making it a static number  $10^{-x}$ ,  $x \geq 1$ , just so it doesn't affect the calculations a lot but still prevents singularity, see what happens, and tweak it from there.

———— \*                      \*  
                                 \*\*                      \* —————

## Implementation

### Main loop

I knew I wanted some sort of “professional game loop”, something I had an idea on how to go about it.

Still, I wanted to make something a little different from the usual. I wanted to maybe have multiple simulations running in parallel — threads allow that.

I also wanted an API that resembles a media player, with start, pause and stop “buttons”, “stepping forward” on a paused simulation (so we can control “how much time is passing”). We can pause a simulation and look at what is happening in it.

Wasn't sure about what's the best way to go about managing the  $\Delta t$  generation. The solution was on using the Strategy design pattern.

So I implemented the Loop class, which also is an iterator and a context manager, so we can use it simply like this:

```
with Loop(lambda: fixed_step(1/24)) as loop:
    loop.skip_forward(3)
    for dt in loop:
        print(dt)
```

One decision I had to make is the choosing of the function of the time module that we would use to calculate the time differences. There were various options; `time.monotonic` seemed like the way to go. But I didn't look too much into it and still didn't have the opportunity to test different strategies.

### Velocity Verlet

I tried to understand why we do it that way by reading the Wikipedia page, but I didn't exactly. So I just tried to implement it by looking at the project statement.

My first implementation was a translation to code to what I was seeing:

```
from nbodysim.object import Object, Vec
from . import Backend, Env
from nbodysim import G
import numpy as np

def step(self, dt: float, env: Env) -> Env:
    updated = []

    for i in range(len(env.objects)):
        x, a = env.objects[i]

        x.position += x.velocity * dt + (1 / 2) * a * dt**2
        x.velocity += (1 / 2) * a * dt

    def acceleration(y: Object) -> Vec:
        diff = y.position - x.position
```

```

    return (y.mass * diff) / ( (np.linalg.norm(diff) ** 2 + env.epsilon**2) ** (3 /
2) )

    a = np.zeros_like(x.position)
    for j, (y, _) in enumerate(env.objects):
        if j == i:
            continue

        a += acceleration(y)
    a *= G

    x.velocity += (1 / 2) * a * dt

    updated.append((x, a))

env.objects = updated
return env

```

Then I noticed that, by not updating all positions and velocities all at once (right now we are doing one by one with the iterator), it's like it actually passed  $n * \Delta t$  time, and only one object updates for each  $\Delta t$ , already influencing the others' acceleration. They should update all at the same time.

A solution is to simply save all initial positions (actually the current objects state) in an array like `python objects = [o for o, _ in env.objects]`, and use that specifically for calculations.

## NumPy

Finally, I learned how to actually use the numpy module, not just to store values. I started by reading the documentation.

After that, I still didn't understand how should I be using it in this situation about the acceleration. My idea was that I was not supposed to be using loops for the calculations.

```

def step(self, dt: float, env: Env) -> Env:
    objs = [obj for obj, _ in env.objects]
    a = np.array([a for _, a in env.objects])

    r = np.stack([o.position for o in objs])
    v = np.stack([o.velocity for o in objs])
    m = np.array([o.mass for o in objs])

    r += v * dt + (1 / 2) * a * dt**2
    v += (1 / 2) * a * dt

    # TODO: calculate acceleration
    # a = G * ...

    v += (1 / 2) * a * dt

    for i, o in enumerate(objs):
        o.position = r[i]
        o.velocity = v[i]

    env.objects = list(zip(objs, a))
    return env

```

With the help of generative AI, I found out how to go about it using matrix multiplication and manipulation, specially by increasing the dimensions of certain arrays/matrixes.



Here is an example, where we take a  $[N, \text{dim}]$  array of vectors of the position ( $N$  is the number of objects and  $\text{dim}$  the dimensions the simulation will be working on) and create a  $[N, N, \text{dim}]$  where each position is the difference of the positions of two objects:

```
diff = r[np.newaxis, :, :] - r[:, np.newaxis, :]
```

I tried commenting this part of the code so it becomes easier to understand what is happening.

## CUDA

Using the example code given by the project statement, I did figure out what to do in the CUDA kernel pretty easy. But I got a little stuck on how exactly to use PyCuda now, specially on how to pass, my data, from host to device *et vice versa*. Generative AI did guide me:

- I need to ravel/flatten the arrays so that CUDA can read them with copy;
- float32 is way faster than float64,

## Conclusion

## Appendices

### Dependencies and Modules discovery

I'm not very familiar with the Python ecosystem.

I already knew about creating a python virtual environment to then install the packages with pip, create the `requirements.txt`. But when I started searching what a common project structure for Python looks like, I learned about the `pyproject.toml` file standard. Then I don't know exactly how it came to be, but I ended up in Scientific Python which introduced me to Pixi which I decided to try out.

One website that I always come back to when I want to view the basics of a programming language, skill, etc., is <https://roadmap.sh/>. I went there to find what were the standard/builtin modules that Python provided and that could be important for me to know. Ended up in Python's complete documentation.

### Modules

The module threading allows for parallelism using threads. It's used to allow multiple Loops to run simultaneously, meaning that we can have multiple simulations running at the same time.

We also get to use quite a lot the `threading.Event` class that allows as to control the Loop with a media player controllers-like API.

We use a queue data structure for a FIFO way of storing the differences of time (implemented by the `queue` module).

The `time` module lets us calculate the differences of time.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Quam ipsum accusantium rem et similique in enim est. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete.

## Bibliography

- [1] "Wikipedia, the Free Encyclopedia." Accessed: Apr. 13, 2026. [Online]. Available: [https://en.wikipedia.org/wiki/Main\\_Page](https://en.wikipedia.org/wiki/Main_Page)
- [2] "Crash Course Physics (YouTube Playlist)." Accessed: Apr. 13, 2026. [Online]. Available: [https://www.youtube.com/playlist?list=PL8dPuuaLjXtN0ge7yDk\\_UA0ldZJdhwkoV](https://www.youtube.com/playlist?list=PL8dPuuaLjXtN0ge7yDk_UA0ldZJdhwkoV)
- [3] "Motion in a Straight Line: Crash Course Physics #1 (Video)." Accessed: Apr. 13, 2026. [Online]. Available: <https://www.youtube.com/watch?v=ZM8ECpBuQYE>
- [4] "Velocity." Accessed: Apr. 13, 2026. [Online]. Available: <https://en.wikipedia.org/wiki/Velocity>
- [5] "Derivatives: Crash Course Physics #2 (Video)." Accessed: Apr. 13, 2026. [Online]. Available: <https://www.youtube.com/watch?v=ObHJJYvu3RE>
- [6] "Integrals: Crash Course Physics #3 (Video)." Accessed: Apr. 13, 2026. [Online]. Available: <https://www.youtube.com/watch?v=ObHJJYvu3RE>
- [7] "Motion in a Straight Line: Crash Course Physics #1," Physics. Accessed: Apr. 13, 2026. [Online]. Available: <https://thecrashcourse.com/courses/motion-in-a-straight-line-crash-course-physics-1/>
- [8] "Newton's Law of Universal Gravitation." Accessed: Apr. 13, 2026. [Online]. Available: [https://en.wikipedia.org/wiki/Newton%27s\\_law\\_of\\_universal\\_gravitation](https://en.wikipedia.org/wiki/Newton%27s_law_of_universal_gravitation)
- [9] "Newtonian Gravity: Crash Course Physics #8 (Video)." Accessed: Apr. 13, 2026. [Online]. Available: <https://www.youtube.com/watch?v=7gf6YpdvtE0>
- [10] "Gravitational Constant." Accessed: Apr. 13, 2026. [Online]. Available: [https://en.wikipedia.org/wiki/Gravitational\\_constant](https://en.wikipedia.org/wiki/Gravitational_constant)
- [11] "Earth Radius." Accessed: Apr. 13, 2026. [Online]. Available: [https://en.wikipedia.org/wiki/Earth\\_radius](https://en.wikipedia.org/wiki/Earth_radius)
- [12] "N-body Problem." Accessed: Apr. 13, 2026. [Online]. Available: [https://en.wikipedia.org/wiki/N-body\\_problem](https://en.wikipedia.org/wiki/N-body_problem)